

Hashing: rehashing e complexidade amortizada

Aplicações: conjuntos, dicionários, contagem de frequências

Prof. Dr. André Yoshiaki Kashiwabara

UFS – COMP0497 Estruturas de Dados

31 de maio de 2026

Recapitulação

Rehashing

Análise amortizada

Pior caso vs. amortizado

Aplicações

Considerações práticas

Recapitulação

- Função hash $h : U \rightarrow \{0, \dots, m - 1\}$.
- Tratamento de colisões: **encadeamento** e **endereçamento aberto** (sondagem linear, quadrática, hash duplo).
- Fator de carga $\alpha = n/m$.
- Custo *esperado* de busca:
 - Encadeamento: $\Theta(1 + \alpha)$.
 - Endereçamento aberto: $\Theta(1/(1 - \alpha))$ (uniform hashing).
- Quando α cresce, o desempenho **degrada**.

Cenário

Inserimos um milhão de chaves em uma tabela de tamanho 1024. Qual o problema?

- $\alpha \approx 977$. Buscas lineares em listas enormes.
- Encadeamento: cada bucket vira uma lista de ~ 1000 elementos.
- Endereçamento aberto: tabela cheia $\Rightarrow$ inserção *impossível*.

Solução: crescer a tabela dinamicamente. Mas como, sem perder a localização das chaves já inseridas?

Rehashing

- Quando α ultrapassa um **limiar** $\alpha_{\max}$, criamos uma nova tabela T' com tamanho $m' > m$.
- Para cada chave k em T :
 - Recalcular $h'(k) \bmod m'$.
 - Inserir em T' .
- Liberar a tabela antiga.

Pontos críticos:

- Não basta copiar: as posições mudam porque m mudou.
- Custo de uma operação que dispara rehash: $\Theta(n)$.
- Operações “normais”: $\Theta(1)$.
- Como conciliar?

- **Crescimento aditivo** ($m' = m + c$): péssimo. Discutiremos por quê.
- **Crescimento multiplicativo** ($m' = 2m$, ou fator $1,5\times$): padrão de mercado.
- **Tamanho primo**: útil para reduzir colisões patológicas; escolhe-se o próximo primo após $2m$.
- **Encolhimento**: ao remover muitas chaves, podemos reduzir m se α cair abaixo de outro limiar (com cuidado: ver *thrashing*).

- Limiar de crescimento típico: $\alpha_{up} = 0,75$ (Java HashMap), 0,66 (Python dict), 0,5 (endereçoamento aberto agressivo).
- Se reduzirmos quando $\alpha < \alpha_{up}/2$, evitamos *thrashing*: sequência alternada de *insert/remove* disparando rehash a cada operação.
- Use sempre uma **histerese** entre os limiares.

Análise amortizada

- Operações isoladas variam:
 - Maioria: $\Theta(1)$.
 - Algumas: $\Theta(n)$ (rehash).
- No *pior caso* por operação, $\Theta(n)$.
- No *caso médio sobre uma sequência* de operações, queremos algo melhor.
- **Análise amortizada:** custo *médio por operação* sobre uma sequência de n operações no pior caso.

1. **Análise agregada:** somar custos totais, dividir por n .
2. **Método contábil (*accounting*):** cobrar mais por algumas operações para “pagar adiantado” o custo de outras.
3. **Método potencial:** associar a cada estado uma função Φ , e escrever o custo amortizado como $\hat{c}_i = c_i + \Phi_i - \Phi_{i-1}$.

Vamos aplicar a primeira ao crescimento de tabelas.

- Começamos com $m_0 = 1$. Inserimos n chaves seguidas.
- A tabela dobra quando α atinge 1: tamanhos 1, 2, 4, 8, ...
- Quantos rehashes? $\lceil \log_2 n \rceil$.
- Custo de cada rehash: copiar todas as chaves atuais.
- Soma:

$$\sum_{i=0}^{\lceil \log_2 n \rceil} 2^i = 2^{\lceil \log_2 n \rceil + 1} - 1 < 2n.$$

- Custo total das inserções: $n + 2n = 3n$.
- **Custo amortizado:** $3n/n = O(1)$ por inserção.

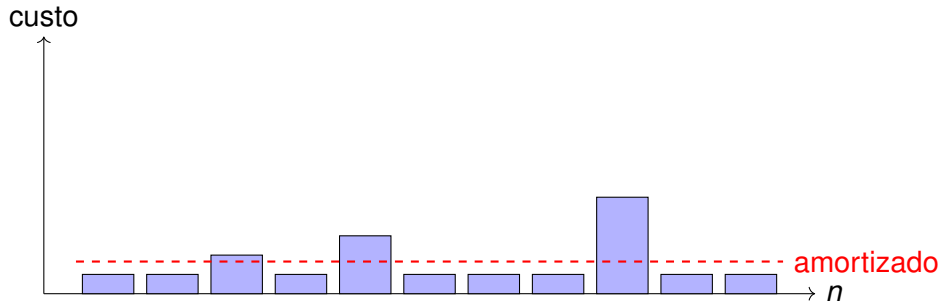
- Suponha $m' = m + c$, c constante.
- Inserir n chaves $\Rightarrow n/c$ rehashes.
- Custos cumulativos: $c, 2c, 3c, \dots, n$.
- Soma: $\Theta(n^2/c) = \Theta(n^2)$.
- **Custo amortizado:** $\Theta(n)$ por inserção.

Moral: dobrar (ou multiplicar por constante > 1) é o que torna **rehashing** viável.

- Seja T_i o estado da tabela após operação i , com n_i elementos e tamanho m_i .
- Função potencial: $\Phi(T) = 2n - m$ (quando $n \geq m/2$).
- Custo amortizado de inserção:
 - Sem rehash: $\hat{c} = 1 + \Delta\Phi = 1 + 2 = 3$.
 - Com rehash ($n_{i-1} = m_{i-1}$, $m_i = 2m_{i-1}$):

$$\hat{c} = (n_{i-1} + 1) + \Delta\Phi = (n + 1) + (2 - n) = 3.$$

- **Conclusão:** cada inserção custa, amortizadamente, $O(1)$.



- Picos em $n = 2, 4, 8, \dots$ — momentos de rehash.
- Linha tracejada: custo amortizado constante.
- “Pague três por inserção”: dois ficam de *crédito* para futuro rehash.

- Se reduzirmos quando $\alpha < 1/2$: pode ocorrer *thrashing*.
- Solução: reduzir só quando $\alpha < 1/4$ (e crescer quando $\alpha = 1$).
- Função potencial híbrida:

$$\Phi = \begin{cases} 2n - m & \text{se } \alpha \geq 1/2 \\ m/2 - n & \text{se } \alpha < 1/2 \end{cases}$$

- Análise (Cormen 17.4) mostra: **inserção e remoção custam $O(1)$ amortizado.**

Pior caso vs. amortizado

- Sistemas **tempo real**: latência de uma única operação importa.
- Servidores web sob latência cauda (*tail latency*): um rehash de 10^7 chaves pausa o serviço por dezenas de ms.
- **Rehashing incremental**: amortizar o trabalho ao longo de várias operações.

- Manter *duas* tabelas em paralelo: T_{antiga} e T_{nova} .
- Cada operação:
 - Move k chaves de T_{antiga} para T_{nova} .
 - Realiza a operação, consultando ambas se necessário.
- Inserções vão direto em T_{nova} .
- Quando T_{antiga} fica vazia, é descartada.
- **Custo por operação:** $O(k)$ no pior caso.
- Usado em Redis, alguns kernels e bancos de dados em memória.

Aplicações

- Operação esperada $O(1)$: `add`, `contains`, `remove`.
- Implementação: tabela hash com chaves apenas (sem valor associado).
- Operações de conjunto: união, interseção, diferença.
- Aplicações: deduplicação, controle de visitados em grafos, filtros de spam (*bloom filters*: hashing probabilístico).

- Estrutura mais usada em programação moderna.
- Java: `HashMap` (encadeamento → árvore quando bucket cresce muito, desde Java 8).
- Python: `dict` (endereçamento aberto, sondagem perturbada, preserva ordem de inserção desde 3.7).
- C++: `std::unordered_map` (encadeamento).
- Aplicações: índices, caches, *memoization*, contagem.

```
1 from collections import defaultdict
2
3 def contar(palavras):
4     freq = defaultdict(int)
5     for p in palavras:
6         freq[p] += 1
7     return freq
```

- Tempo total: $\Theta(N)$ esperado, onde N = número de palavras.
- Tempo com BST: $\Theta(N \log U)$, onde U = vocabulário.
- Hash vence quando ordenação não é necessária.
- Aplicação clássica: *word count* em logs, MapReduce, NLP.

- **Caches** (LRU): hash + lista duplamente ligada $\Rightarrow O(1)$ para get e put.
- **Memoization**: cachear resultados de função pura.
- **Symbol tables** em compiladores.
- **Detecção de duplicatas** em streams.
- **Hashing criptográfico**: SHA-256, integridade de arquivos (uso diferente: resistência a colisão deliberada).
- **Hashing consistente**: balanceamento em sistemas distribuídos (caches, sharding).

Considerações práticas

- Boa função: distribuição quase-uniforme sobre $\{0, \dots, m - 1\}$.
- Strings: *polynomial rolling*, FNV, MurmurHash, SipHash.
- Inteiros: multiplicativo de Knuth ($h(k) = \lfloor m(k\phi \bmod 1) \rfloor$).
- Em ambientes adversariais (entrada controlada por atacante), use hash com **semente aleatória** (ex. SipHash em Python, Rust).
- **Evite**: divisão por m não-primo com chaves correlacionadas.

- 2003: ataque a Perl, PHP, Java; sequências escolhidas geram colisões em cascata.
- Sintoma: requisição HTTP com 1000 parâmetros causa *denial of service*.
- Mitigação: **hash com chave secreta** (SipHash, randomização de seed por execução).
- Java 8 mitiga via *treeify*: bucket vira árvore vermelho-preta quando excede 8 elementos.

- **Rehashing** permite tabelas hash dinâmicas sem perder $O(1)$ esperado por operação.
- **Crescimento multiplicativo** é essencial: aditivo dá $\Theta(n)$ amortizado.
- **Análise amortizada** formaliza o custo médio sobre sequências: agregada, contábil, potencial.
- Para sistemas *soft real-time*, prefira **rehashing incremental**.
- Aplicações dominam programação moderna: conjuntos, dicionários, contagem, caches.

- Implementar uma tabela hash com rehashing automático em C.
- Medir tempo por operação em uma sequência de 10^6 inserções.
- Comparar com `std::unordered_map`.
- Refletir: como você projetaria um *dict* para um sistema de trading com latência cauda < 1 ms?

Obrigado!